Objetivo:
Implementar y dejar funcionales en *ambiente de prueba* los módulos que corresponden a la entrega intermedia de este proyecto: **Ventas, Almacén, Tesorería básica, Gastos, Reportes y Bot de pedidos**, asegurando que **todas las piezas estén integradas entre sí**, con pruebas básicas, migraciones, endpoints y criterios de aceptación claros.

---

1) Requisitos generales (aplicar a todo)

- Trabajar sobre la rama feature/hito-intermedio y dejar PR listo para revisión.
- Mantener coherencia con arquitectura existente (Server Actions, Supabase, RPC, Zod).
- Crear migraciones SQL para cualquier tabla nueva o alteración; incluir rollback.
- Implementar controles de transacción en operaciones críticas (crear pedido + descontar stock + registrar movimiento de caja).
- Añadir tests básicos (unitarios/integ) o scripts de verificación (curl/postman) para cada flujo.
- Documentar en README.md del proyecto: endpoints nuevos, RPCs, eventos y cómo probar en ambiente de prueba.

---

2) Base de datos (Supabase/Postgres) — esquema y migraciones

- **Tablas nuevas / modificaciones** (si ya existen, adaptar; si no, crear):
  - tesoreria_cajas (id, sucursal_id, nombre, saldo_inicial, saldo_actual, moneda, created_at, updated_at)
  - tesoreria_movimientos (id, caja_id, tipo [ingreso/egreso], monto, descripcion, origen_tipo [venta/gasto/adelanto], origen_id, user_id, created_at)
  - gastos (id, sucursal_id, categoria_id, monto, comprobante_url, descripcion, fecha, creado_por, created_at)
  - gastos_categorias (id, nombre, descripcion)
  - cuentas_corrientes (id, cliente_id, saldo, limite_credito, updated_at)
  - cuentas_movimientos (id, cuenta_corriente_id, tipo [cargo/pago], monto, descripcion, fecha, origen_tipo, origen_id)
  - reportes_export (registro para auditoría de exportaciones si hace falta)
- **Alteraciones**:
  - En tabla pedidos: agregar pago_estado (pendiente, parcial, pagado), total, caja_movimiento_id (nullable).
  - En tabla clientes: asegurar limite_credito y bloqueado_por_deuda boolean.
- **Funciones / RPC necesarias**:
  - fn_crear_movimiento_caja(caja_id, tipo, monto, descripcion, origen_tipo, origen_id, user_id) — registra movimiento y actualiza saldo (en transacción).
  - fn_registrar_gasto(sucursal_id, categoria_id, monto, comprobante_url, descripcion, creado_por) — registra gasto y opcionalmente crea movimiento de caja.
  - fn_crear_pago_pedido(pedido_id, caja_id, monto, tipo_pago) — aplica pago sobre pedido, actualiza cuentas corrientes si aplica.
  - fn_consultar_stock_por_lote(producto_id) — ya existe, asegurar que retorne lotes disponibles.
  - Asegurar fn_crear_pedido_bot() actualizada para generar total, crear movimientos de caja si el cliente paga por adelantado, y/o registrar deuda en cuentas_corrientes.
- **RLS/Permisos**: mantener reglas actuales; documentar cualquier permiso nuevo.

---

3) Lógica transaccional y casos de uso clave

Implementar flujos atómicos (DB transaction) para:

Flujo A — Crear pedido desde web / admin

1. Validar stock (por lotes FIFO).
2. Reservar lotes (marcar como reservado o crear movimiento de stock tipo salida pendiente).
3. Crear pedido (pedidos) con total.
4. Si pago inmediato: crear movimiento de caja con fn_crear_movimiento_caja y registrar pago_estado.
5. Si crédito: registrar cargo en cuentas_movimientos y actualizar cuentas_corrientes.saldo.
6. Notificar (webhook/evento) al bot/reparto si corresponde.

Flujo B — Bot de pedidos (WhatsApp)

1. Mantener validaciones actuales; al confirmar pedido ejecutar el mismo flujo transaccional que Flujo A (usar RPC para garantizar atomicidad).
2. Si el cliente indica pago en efectivo al recibir (cobro repartidor), crear pedido con pago_estado = pendiente y generar instrucción para que repartidor registre movimiento de caja en app PWA (en este hito la PWA puede ser pendiente; registrar como acción en ruta).

Flujo C — Tesorería básica (cajas)

- Creación de cajas por sucursal.
- Movimientos: ingresar cobros de ventas, egresos por gastos, cierre de caja diario.
- Conciliación: endpoint para listar movimientos por caja y por fecha.
- Al registrar venta con cobro: fn_crear_movimiento_caja debe ligarse al pedido.caja_movimiento_id.

Flujo D — Registro de gastos

- Interfaz para cargar gasto con adjunto (URL a storage).
- Al guardar gasto, opción: "afectar caja" → crea tesoreria_movimientos tipo egreso.
- Reportes: poder filtrar por categoría, sucursal, fecha.

Flujo E — Cuentas corrientes

- Al crear pedido con crédito: registrar cuentas_movimientos (cargo).
- Al registrar pago (manual o automático), debitar cuentas_movimientos y crear movimiento de caja.
- Lógica de bloqueo: si cuentas_corrientes.saldo > limite_credito → marcar cliente bloqueado_por_deuda = true y bloquear venta en validación.

---

4) Endpoints / Server Actions / UI a crear o completar

- **Server Actions / API routes**:
  - /api/tesoreria/cajas (GET/POST/PUT)
  - /api/tesoreria/movimientos (GET/POST)
  - /api/gastos (GET/POST)
  - /api/reportes/export (POST) — genera export (CSV/PDF) según filtros
  - /api/cuentas_corrientes (GET/POST/payments)
  - Actualizar /app/api/bot/route.ts para invocar RPC de pagos y movimientos cuando aplique.

- **Páginas / UI** (admin):
  - /tesoreria/cajas — listado, crear caja, ver saldo y movimientos.
  - /tesoreria/movimientos — crear ingreso/egreso manual (form con validación Zod).
  - /gastos — listado, crear gasto con adjunto (upload a Supabase Storage).
  - /reportes — UI para generar reportes y botones de exportar CSV / PDF (usar server-side para generación).
  - Actualizar /ventas/pedidos/[id] — vista detalle: mostrar sección "Tesorería" con pagos vinculados, botones "Registrar pago" (efectivo/transferencia).
  - Validación en formularios de ventas: bloquear venta si cliente bloqueado_por_deuda.
- **Componentes UI**:
  - Reutilizar TanStack Table para listados, React Hook Form + Zod en forms.
  - Usar Cards para resúmenes en Dashboard (saldo cajas, gastos recientes, cuentas pendientes).
- **Reportes/Export**:
  - Implementar export CSV para: Ventas por fecha, Gastos por categoría, Movimientos de caja, Cuentas corrientes (saldos).
  - Endpoint que arma CSV en servidor y devuelve URL temporal o descarga.

---

5) Integraciones específicas

- **Bot (WhatsApp / Twilio)**:
  - Al confirmar pedido con pago adelantado → crear movimiento de caja (RPC) y enviar comprobante (si corresponde).
  - Si cliente consulta deuda → bot responde con saldo obtenido de cuentas_corrientes.
  - Si cliente quiere pagar vía link → generar referencia de pago (puede ser placeholder) y marcar pago_estado al confirmarse (en este hito basta con crear la referencia y endpoint para marcar pago manualmente).
- **Supabase Storage**:
  - Uso para adjuntar comprobantes de gastos y facturas. Validar permisos y rutas.

---

6) Validación y criterios de aceptación (QA)

Para considerar el hito cumplido, probar y demostrar (logs/capturas/PR) lo siguiente en ambiente de prueba:

1. Crear caja en sucursal A y ver saldo inicial.
2. Generar un pedido desde web por producto con stock: stock descontado por lotes FIFO, pedido creado con total.
3. Registrar pago en caja por ese pedido: movimiento de caja creado, saldo actualizado y pedido pago_estado = pagado.
4. Generar otro pedido con crédito: cuentas_corrientes recibe cargo, cliente.saldo actualizado; intentar crear pedido que exceda limite_credito y bloquear.
5. Registrar gasto con adjunto y marcarlo como afectando caja: movimiento de caja egreso creado.
6. El bot hace pedido y, al confirmar, ejecuta el mismo flujo transaccional que web (stock + pedido + posible movimiento de caja).
7. Generar un reporte (Ventas por fecha) y descargar CSV que contenga ventas con totales y estado de pago.
8. Dashboard muestra saldo de cajas, gastos recientes y cuentas pendientes (datos reales desde BD, no mock).

---

7) Tareas extras recomendadas (prioridad baja para este hito)

- Notificaciones por email/WhatsApp cuando cliente quede bloqueado_por_deuda.
- Historial de conciliación de depósitos (matching por número de operación).
- Endpoint para conciliación automática por importación CSV bancario.

---

8) Entregables esperados (al terminar)

- Branch feature/hito-intermedio con commits claros.
- PR con descripción y checklist de aceptación.
- Migraciones SQL y scripts de rollback.
- Endpoints y Server Actions implementados.
- UI pages y forms funcionales en ambiente de prueba.
- Documentación en README sobre cómo probar cada flujo.
- Listado de pruebas exitosas (screenshots / logs / curl responses).

---

9) Nota técnica final (importante)

Todas las operaciones que toquen **stock, pedido y caja** deben ejecutarse en una misma transacción atómica (usar RPC o transaction blocks en Postgres), para evitar inconsistencias. Cualquier operación que afecte cuentas_corrientes debe generar un cuentas_movimientos y opcionalmente un tesoreria_movimiento si hay pago real.